

Assignment – 5

Date: 8/9/2026

1. WAP to consider a base class Notification with a method send().

Create a derived class EmailNotification that provides its own send() implementation.

Inside the child class, demonstrate two different ways of invoking send():

- Calling the method through the current object.
- Calling the parent implementation through the inheritance relationship.
[self.send() vs super().send()]

Must follow:

1. Implement both approaches.
2. Observe and explain the difference in their behavior.
3. Be careful to identify what happens if the child method calls itself indirectly through the object again.

2. WAP for the training organization, which has the following relationship:

Classes: 'Employee', 'Developer' (child class of Employee), 'Trainer' (child class of Employee), 'TechMentor' (child of Developer and Trainer).

Employee contains a common method work().

Both Developer and Trainer provide their own version of work().

TechMentor inherits from both Developer and Trainer.

Each implementation should use the inheritance chain appropriately so that Python can determine the order in which methods are executed. []

Task:

1. Implement the complete hierarchy.
2. Create a TechMentor object.
3. Call work().
4. Print the MRO of TechMentor.
5. Explain why Python does not simply execute the methods based on the visual left-to-right structure of the diagram.

3. Integrated Inheritance Challenge:

WAP for Notification Framework: A software company is developing a notification framework that supports multiple communication channels.

- Also draw a class hierarchy: Design a notification system in which 'Notification' acts as the common base class. Create two classes, 'Message' and 'Alert', that inherit from Notification. Further, create 'Email' and 'SMS' as specialized classes derived from 'Message', and create 'SystemAlert' as a specialized class derived from 'Alert'. Finally, create an 'EmailSMS' class that inherits from both 'Email' and 'SMS' to combine their functionalities.
-

Notification and design the following system:

- ✓ The base class should contain common notification information.
- ✓ Message and Alert should introduce their own behavior.
- ✓ Email and SMS should specialize in message delivery.
- ✓ SystemAlert should specialize alert behavior.
- ✓ EmailSMS should combine the capabilities of Email and SMS.

Several classes should contain methods with the same name so that the behavior of method overriding and inheritance can be observed.

Use the inheritance mechanism so that parent behavior can participate without manually creating parent objects.

Task:

1. Design the complete class hierarchy.
2. Create an EmailSMS object.
3. Call the common operation through the child object.
4. Observe which implementations execute.
5. Display the MRO of EmailSMS.
6. Compare the result of calling a method through the object with explicitly accessing a parent implementation through super().
7. Explain the inheritance type represented by the overall design.
8. Identify the potential diamond problem in the hierarchy and explain how Python's MRO handles it.